

LibreOffice Calc Add-In — NBA Data

Claude Code build prompt · balldontlie API · Java 8 / dependency-free

Goal

Build a free, open-source (MIT License) LibreOffice Calc add-in that exposes NBA teams, games, standings, player-search, and player/game statistics as spreadsheet functions, sourced from the **balldontlie** API. The add-in must run inside the LibreOffice-embedded JVM and follow the same architecture as my prior Calc add-ins.

Critical API facts

Verified July 2026 — do not assume the old keyless behavior.

balldontlie now requires a free API key on every request.

Passed as an HTTP header: Authorization: <API_KEY> (raw key, no Bearer prefix). There is no keyless access anymore.

- Base URL: `https://api.balldontlie.io`
- NBA endpoints are versioned under `/nba/v1/` (plus a few legacy `/v1/` paths). Make the version path a constant so it's easy to change.
- Cursor-based pagination via `meta.next_cursor`; request next page with `?cursor=NEXT_CURSOR`. `per_page` defaults to 25, max 100.
- Free tier has real rate limits (a few req/min). Handle HTTP 429 gracefully with backoff.
- Data range: 1946–current.

Endpoints to support

GET <code>/nba/v1/teams</code>	all teams
GET <code>/nba/v1/players?search=<name></code>	player search (paginated)
GET <code>/nba/v1/games?dates[]=YYYY-MM-DD</code>	games / scores
GET <code>/nba/v1/games?seasons[]=YYYY&team_ids[]=<id></code>	
GET <code>/nba/v1/standings?season=YYYY</code>	standings
GET <code>/nba/v1/stats?game_ids[]=<id></code>	per-game box stats
GET <code>/nba/v1/stats?seasons[]=YYYY&player_ids[]=<id></code>	
GET <code>/nba/v1/season_averages/...</code>	(optional; may need paid tier)

Query-array params use the PHP-style `key[]=value` repeated-key convention. Multiple values = repeat the key. URL-encode everything.

API key handling

Cell functions must never contain the key as a literal. Resolve the key in this order:

- Java system property `balldontlie.apikey`

- Environment variable `BALLDONTLIE_API_KEY`
- Properties file at `~/.config/libreoffice-nba/balldontlie.properties` containing `apiKey=...`

If no key is found, functions return a clear sentinel string like `#NO_API_KEY` (not an exception, not a silent blank) so the user sees what's wrong in the cell. Document all three mechanisms in the README.

Architecture constraints

LibreOffice-embedded JVM — hard requirements from prior add-ins; do not deviate.

Language & dependencies

- **Java 8 source/target.** No records, no var, no `java.net.http.HttpClient`, no switch expressions, no text blocks. Use `HttpURLConnection` for networking.
- **Zero third-party dependencies.** No Jackson/Gson/Apache HTTP. Hand-roll a minimal, tolerant JSON parser sufficient for these responses. Bundled deps risk classloader conflicts inside the LibreOffice JVM.
- **Add-In only pattern** — implement `XAddIn` + `XServiceName/XServiceInfo`, registered via `.oxt`.

CalcAddIn.xcu MUST set CompatibilityName for every function.

Missing this causes silent data loss when the sheet is saved as XLS/XLSX and reopened — a recurring critical bug. Verify each function has both the programmatic name and a `CompatibilityName`.

Non-blocking cell functions

Cell functions must never block on network I/O. Use the cache + background-fetch + sentinel-return pattern:

- On first call for a key, return a sentinel like `#LOADING` (or cached stale value), and kick off an async fetch on a background thread.
- When the fetch completes, store into the cache; the value appears on the next recalc (F9 / Ctrl+Shift+F9). Document this recalc behavior in the README.
- Cache keyed by the full normalized request (endpoint + sorted params), with configurable TTL (scores/games 5 min, standings 1 hr, teams 24 hr).
- Thread-safe cache (`ConcurrentHashMap`) with bounded size / simple eviction.
- Deduplicate in-flight requests so identical concurrent calls fetch once.
- Handle HTTP 429 and 5xx with bounded exponential backoff on the background thread. Surface persistent failures as `#ERR` sentinels; expose last error via `NBA_LASTERROR()`.

Functions to expose

All with `CompatibilityName` set. Multi-row/col results return a 2-D array (sequence-of-sequences) so they spill into a range; single-value convenience functions too.

Function	Returns
<code>NBA_TEAMID(name abbrev)</code>	numeric team id

NBA_TEAMS()	2-D table of all teams
NBA_PLAYERSEARCH(name)	id, first, last, position, team abbrev
NBA_PLAYERID(fullName)	single best-match player id
NBA_GAMES(dateOrSeason, [teamId])	games w/ scores & status
NBA_SCORE(season, abbrev, [nth])	most-recent (or Nth) result string
NBA_STANDINGS(season, [conf])	2-D standings table
NBA_TEAMRANK(season, abbrev)	single conference rank
NBA_PLAYERSTAT(season, id, key)	season-agg or per-game stat
NBA_BOXSCORE(gameId)	per-player box score for a game
NBA_LASTERROR()	last error message (diagnostics)
NBA_CACHECLEAR()	clears cache, returns count cleared

statKey ∈ pts, reb, ast, stl, blk, fg_pct, fg3_pct, ft_pct, min, etc. All functions need XML descriptions and per-parameter descriptions in CalcAddIn.xcu so they appear in the Function Wizard.

Build & packaging

- Plain Java 8, dependency-free. Prefer a simple `build.sh` (javac + jar + zip into `.oxt`) over Maven, matching prior add-ins — a minimal `pom.xml` is acceptable if it pulls no runtime deps and targets Java 8.
- Produce a working `.oxt` installable via `unopkg add` / Extension Manager.
- Put LibreOffice URE/UNO jars on the compile classpath but do not bundle them (host-provided).
- Cross-platform: Slackware, Debian, Ubuntu, Windows. Keep `build.sh` POSIX; note Windows build steps in the README.

Deliverables

- Full Java 8 source tree, including the hand-rolled JSON parser.
- CalcAddIn.xcu with every function + CompatibilityName + wizard descriptions.
- `description.xml`, `manifest.xml`, and the rest of the `.oxt` skeleton.
- `build.sh` (and optional minimal `pom.xml`) that produces the `.oxt`.
- README.md: API-key setup (all 3 mechanisms), install, async/recalc behavior, rate-limit notes, free-tier caveat, function reference with examples, cross-platform build.
- LICENSE (MIT).

Working process

Prompt-first, then code. Before writing code, briefly confirm the architecture decisions above and flag any tradeoff you'd resolve differently — especially:

- JSON parser scope (how tolerant it must be for these specific responses).
- Sentinel-string set and how errors surface.
- Cache TTL defaults and eviction policy.
- How array-returning functions behave in a single cell vs a range.

Once those are settled, generate the complete, compiling add-in.